

Distributed Multi-Model Analytics for E-commerce Data

Diane Musabyimana 101218

[Diane-AUCA/Final-exam_BDA: final exam project of big data analytics](#)

1. Introduction

This report summarizes a distributed multi-model analytics system built for large-scale e-commerce data, combining MongoDB, HBase, and Apache Spark to address different analytical needs within a single ecosystem.

The simulated dataset spans 90 days of activity: 10,000 users, 5,000 products, 25 categories, 500,000 transactions, and 2,000,000 browsing sessions. Each technology was chosen to match the structure and access patterns of a specific data type.

1.1 Project Objectives

- Design and justify data schemas for MongoDB and HBase
- Implement ingestion and querying for both databases
- Develop batch processing and SQL analytics with Apache Spark
- Integrate data across systems for cross-database analytics
- Generate and visualize business insights for decision-making

2. System Architecture

A multi-model architecture was used because no single database performs equally well across all analytical workloads. MongoDB stores flexible transactional documents, HBase handles high-volume time-series session data, and Spark performs distributed processing and cross-source integration. Python scripts orchestrate data generation, loading, and analytics execution.

Component	Role	Data Stored
MongoDB	Document store	Users, products, transactions
HBase	Wide-column store	Sessions, product metrics
Apache Spark	Distributed processing	Cleaning, analytics, CLV
Python scripts	Orchestration	Loading, integration, visualization

Spark integrates data from MongoDB and HBase to run distributed computations such as SQL analytics, revenue analysis, device usage analysis, and product recommendations, separating storage from computation so each technology does what it does best.

3. Data Modeling and Storage

3.1 MongoDB Schema Design

MongoDB's document model was chosen for its natural fit with nested, interconnected e-commerce data. Three collections use deliberate embedding: Users embed a pre-computed purchase_summary (total_orders, total_spent, last_order_date) to avoid runtime aggregation; Products embed category and subcategory data (including profit margin) to avoid joins; and Transactions embed a line-items array, allowing full transaction analysis from a single document read.

Key indexes support performance: transactions.user_id and transactions.timestamp for lookups and time-range queries, transactions.items.product_id for product analysis, products.category.category_id for category aggregations, and users.purchase_summary.total_spent for customer segmentation.

3.2 HBase Schema Design

HBase was chosen for session data due to its strength with high-volume, sparse, time-series records and efficient row-key range scans.

Table	Row Key	Purpose
user_sessions	user_id#timestamp	Groups a user's sessions chronologically; columns cover session info, device, geo, and activity
product_metrics	product_id_date	One row per product per day (419,073 rows from 2M sessions); supports time-range view-count queries

3.3 Technology Selection Summary

Data Type	Store	Reason
Users, products, transactions, categories	MongoDB	Rich nested structure, hierarchical embedding, flexible schema
Sessions, product metrics	HBase	High volume, time-series, sparse columns, range scans

4. MongoDB Aggregation Pipelines

4.1 Revenue Analytics by Category

This pipeline unwinds each transaction's items array, joins to products to retrieve the embedded category name (no second lookup needed, since category data is denormalized), groups by category to sum revenue and units, and returns the top 10 categories by revenue.

Rank	Category	Revenue	Orders	Units Sold
1	Cameron Parsons	\$22,221,952	40,189	80,261
2	Kelly Santiago	\$21,715,954	43,106	86,006
3	Anderson, Walls and Duncan	\$21,434,231	43,421	86,841

Revenue is evenly distributed across categories, reducing dependence on any single product type and lowering business risk.

4.2 User Segmentation by Purchasing Frequency

Segment	Users	Avg Spent	Avg Orders
High Value (55+ orders)	2,574	\$54,958	58.9
Medium Value (45-54 orders)	5,252	\$46,206	49.5
Low Value (1-44 orders)	2,174	\$37,921	40.7

All 10,000 users placed at least 20 orders, reflecting a highly active synthetic user base, with a near-even split between High and Medium Value segments (52% vs 48%).

4.3 Top Selling Products

A separate pipeline ranks products by units sold and revenue (e.g., prod_00283 led with 661 units sold for \$121,352), identifying top performers for inventory and promotion planning.

5. Apache Spark Data Processing

Spark's batch job cleans each dataset before registering it as a SQL view: timestamps are standardized, nested objects (geo_data, device_profile) are flattened, nulls are filled (e.g., unknown location, zero stock), categorical fields are normalized, and duplicates are removed by key.

5.1 Spark SQL Analytics

- Revenue by Category joins transactions and products via a lateral-view explode of the items array.
- Device Usage and Conversion Rates uses conditional aggregation (CASE WHEN) to find which device/OS combinations convert best.
- Cross-Database Join (MongoDB + HBase) joins transaction data with session data on session_id to see which referrers and devices drive the most revenue; this is the key demonstration of Spark SQL's value, since neither MongoDB nor HBase alone can perform this join efficiently.

5.2 Product Recommendation Engine

Using a self-join on the exploded items DataFrame, the pipeline finds product pairs frequently bought together, groups by pair, counts co-occurrences, and returns the top 10 pairs. Co-purchase counts were low (3-4 occurrences), as expected given 5,000 products and randomly generated transactions; in a real system these pairs would power 'customers also bought' recommendations.

6. Integrated Analytics: Customer Lifetime Value

Business question: who are the most valuable customers, and how does browsing engagement relate to spending? MongoDB provides user profiles and transaction totals; HBase provides session counts, durations, and conversions for a 50,000-session sample. The two are joined on user_id, users are classified into CLV tiers by spend, and a ranked list and tier summary are produced.

Tier	Spending Threshold	Users Found	Avg Spent	Avg Orders	Avg Sessions
Gold	\$30,000 - \$49,999	64	\$33,973	37.5	205
Silver	\$10,000 - \$29,999	187	\$24,039	29.9	197

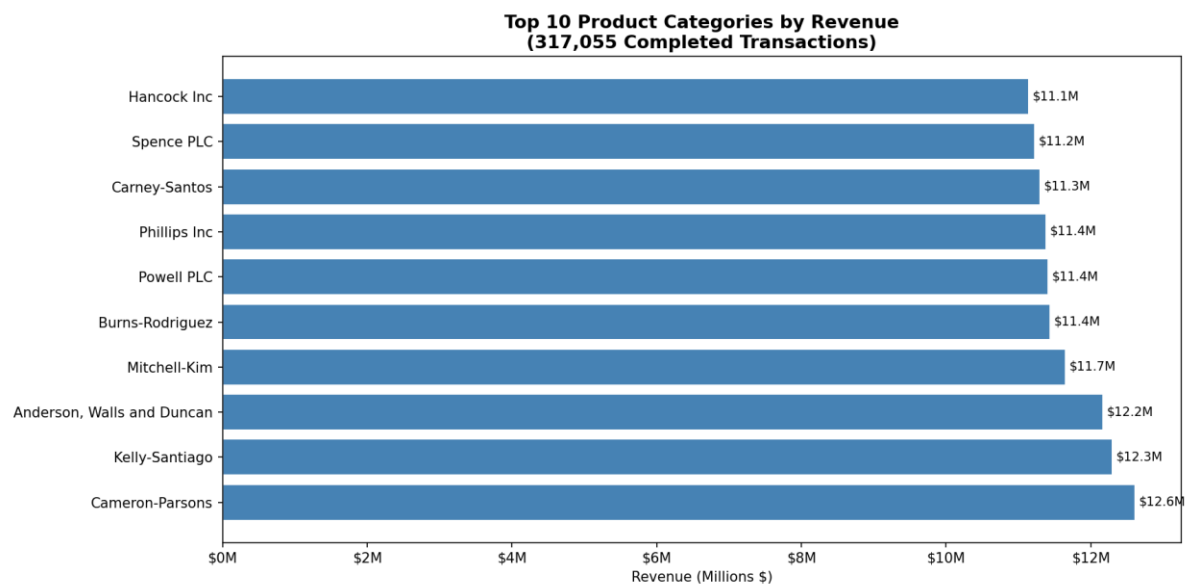
The 251 users with CLV data came from the 50,000-session sample; a full scan of all 2,000,000 sessions would surface all 10,000 users, but the sample is sufficient to demonstrate the integration pattern.

- Gold-tier customers average 205 sessions and 30+ minutes per session highly engaged, deep-exploring browsers.
- The Silver-to-Gold ratio (187 vs 64) suggests an opportunity to promote Silver customers toward Gold tier.
- High spenders are also high engagers, confirming session engagement predicts customer value.

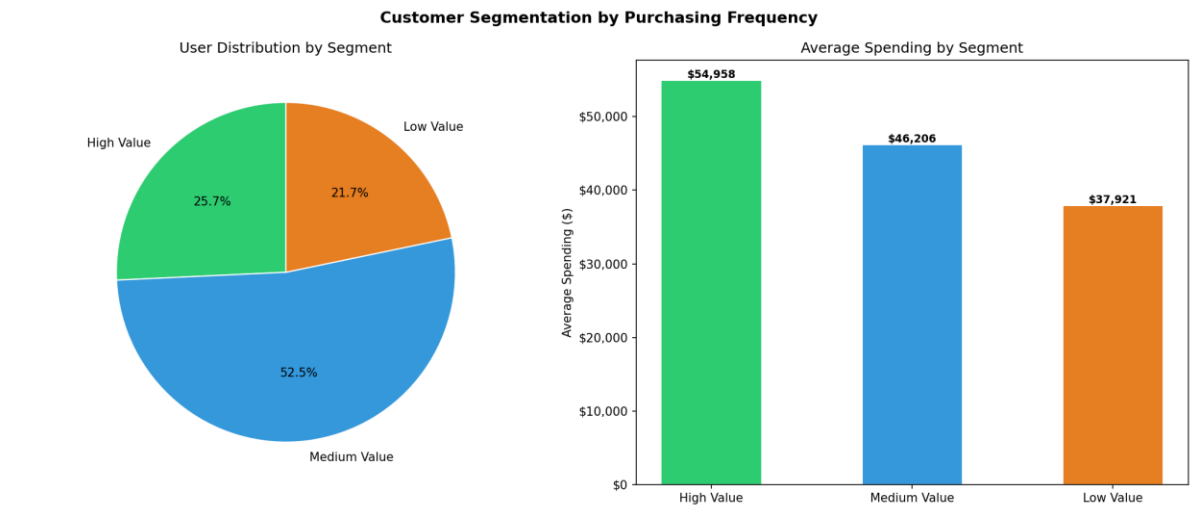
7. Visualizations and Business Insights

Four Matplotlib visualizations addressed different business dimensions of the dataset:

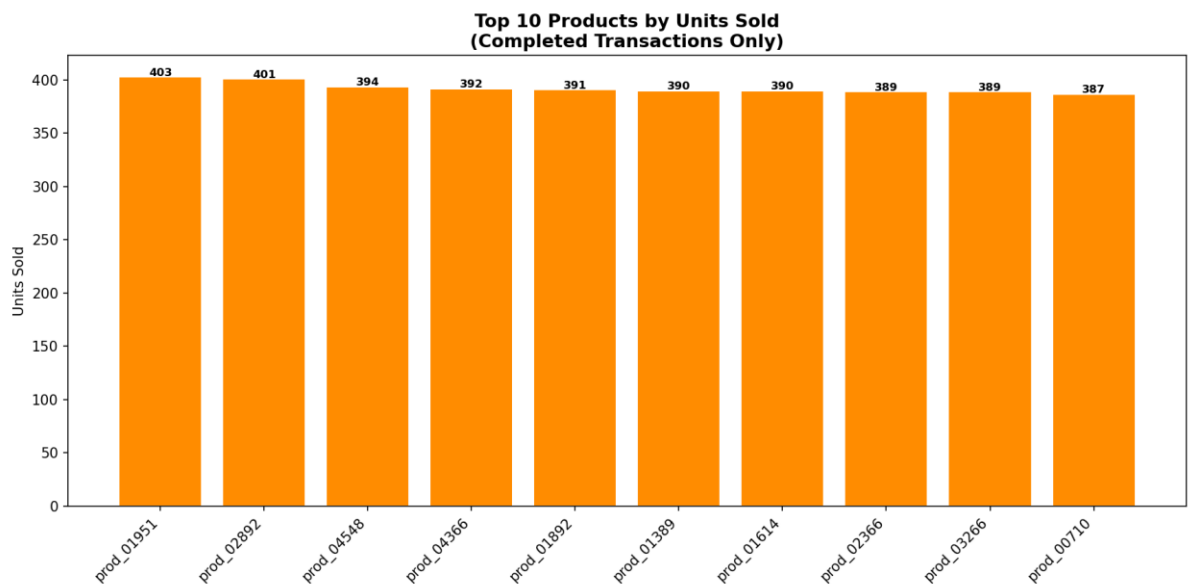
- Revenue by Category (bar chart) top 10 categories each generate \$11M-\$12.6M, a balanced distribution that limits the impact of any single category's failure.



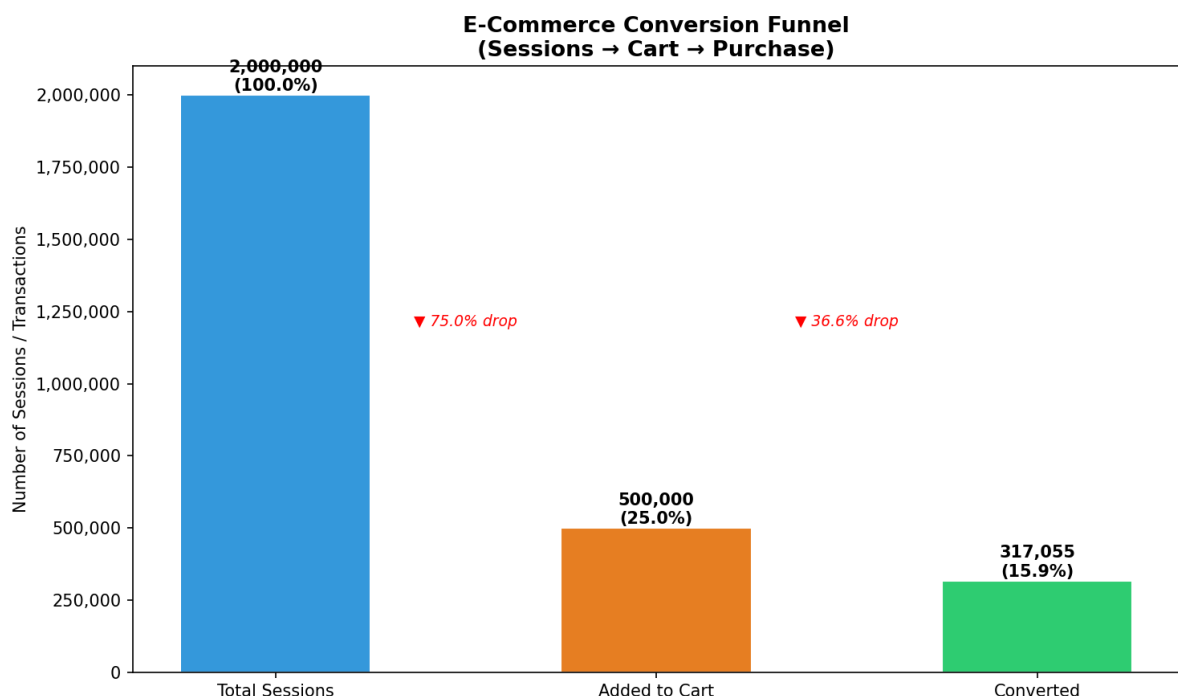
- Customer Segmentation (pie/bar) a near-even High Value (52%) / Medium Value (48%) split, with a \$10,000+ spending gap suggesting an upgrade opportunity via loyalty programs.



- Product Performance (bar chart) top 10 products by units sold, useful for inventory planning and for bundling top sellers with slower-moving items.



- Conversion Funnel tracks sessions through cart additions to purchases; a 25% overall conversion rate is strong for e-commerce, while the gap between cart additions (35%) and conversions (25%) implies roughly a 28% cart-abandonment rate, a target for recovery email campaigns.



•
•

8. Scalability Considerations

MongoDB can shard the transactions collection on `user_id` across nodes, with pre-computed purchase summaries reducing read load at peak traffic. HBase, designed for horizontal scale, would distribute its 2,000,000 session rows across region servers; the `user_id#timestamp` row key avoids hotspots by spreading sessions across users rather than concentrating recent activity on one server. Spark scales by adding worker nodes without code changes, benefits from broadcast joins for smaller DataFrames, and in production would connect directly to MongoDB and HBase via native connectors instead of the Python-level transfer used here.

9. Limitations and Future Work

9.1 Current Limitations

- The CLV analysis was performed using a sample of 50,000 session records instead of the full dataset of approximately 2,000,000 sessions due to local memory and hardware constraints.
- The implemented CLV analysis integrates MongoDB and HBase using Python libraries (pymongo and happybase) rather than a native Spark-HBase connector. Consequently, the analysis is executed on a single machine instead of taking advantage of distributed processing.
- An Apache Spark implementation was developed to perform distributed aggregation and cross-database joins. However, the implementation encountered a Spark runtime environment error ("Python worker exited unexpectedly") during execution. As a result, the Spark-based distributed analysis could not be completed within the project timeframe.

9.2 Future Work

- Implement the native Spark-HBase connector for true distributed session processing.
- Resolve the Spark environment issues to enable successful execution of the Spark-based CLV analysis.
- Use the native Spark-HBase connector for distributed processing instead of Python-based integration.
- Process the full dataset of approximately 2,000,000 session records instead of a 50,000-session sample.

10. Conclusion

This project demonstrates a multi-model analytics architecture that uses each technology where it adds the most value: MongoDB for flexible document storage and aggregation, HBase for high-volume time-series session data, and Spark as the distributed processing and integration layer.

Key business insights include an even revenue distribution across categories, a highly active customer base (all users with 20+ purchases), a strong link between session engagement and customer lifetime value, and a 25% overall conversion rate that offers a clear optimization target.

The central architectural finding is that no single database optimally serves all analytical needs: separating document data (MongoDB) from time-series behavioral data (HBase), unified through Spark, reflects a production-grade approach to big data analytics in e-commerce.